P1161R2
Deprecate uses of the comma operator in subscripting expressions

Published Proposal, 2019-01-21

This version:
https://cor3ntin.github.io/CPPProposals/deprecate_comma_subscript/P0000.html

Author:
Corentin Jabot

Audience:
CWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose to deprecate the use of the comma operator in subscripting expressions as they are not very useful, confusing and limit futures evolutions of the standard.

Table of Contents

§ 1. Revisions

§ 1.1. Revision 2

- Target CWG

§ 1.2. Revision 1

- Improve Wording

§ 2. Introduction

This paper proposes that the use of a comma operator within a subscripting expression be deprecated. Doing so opens the door to multidimensional subscript operators for classes such as `mdspan`.

Current

```
array[x]     // Ok
array[(x,y)] // Ok, uses y as index/key
array[x,y]   // Ok, uses y as index/key
```

Proposed

```
array[x]     // Ok
array[(x,y)] // Ok, uses y as index/key
array[x,y]   // Deprecated, uses y as index/key
```

§ 3. Motivation

Currently, a comma can appear in subscript expressions such that `auto z = foo[x, y]` calls the comma operator with `y` as the argument. While this is currently unambiguous, it is confusing when encountered and error-prone when used.

The authors think this syntax would be more useful and suited to index multidimensional classes such as `mdspan`.

```
mdspan<int, array_property::dimension<2, 3, 5>> foo(/*...*/);
int value = foo[1, 2, 3];
```

We do not propose to make this possible until a reasonable deprecation period has passed.

However [P1277] suggests a shortest depreciation period and provides wording for a multi-dimensional operator subscript.

It is the hope of the authors that `mdspan` will be able to adopt such an operator in a timely fashionn rather than nastily re-purposing the call operator for lack or a more suitable alternative.

It is important to note that the goal is not to special-casing the meaning of a comma within a subscript expression. Indeed, the standard defines that:

> In contexts where comma is given a special meaning, *[Example: in lists of arguments to functions ([expr.call]) and lists of initializers ([dcl.init]) — end example]* the comma operator as described in this subclause can appear only in parentheses.

So simply by supporting multiple parameters, a comma within a subscript operator would serve as an argument separator, without the need for specific wording or parsing.

§ 3.1. The need for a multidimensional subscript operator

The classes that can benefit from a multidimensional subscript operator are higher-dimensions variants of classes that define a single-dimension subscript operator: matrixes, views, geometric entities, graphics APIs...

`mdspan` comes immediately to mind. Note that because there is no multidimensional subscript operator in the standard, `mdspan` uses the call operator. While this is functionally equivalent to what a multidimensional subscript operator would be, it does not carry the same semantic, making the code harder to read and reason about. It also encourages non-semantical operator overloading.

§ 3.2. What a multidimensional subscript operator would look like

This paper does not propose a multidimensional subscript operator, yet, it's interesting to look at what it would look like. The most logical thing to do would be to mirror the call operator syntax.

```
template <typename DataType>
class mdspan {
  using reference = /*...*/;
  template<size_t...>
  reference operator[]( IndexType ... indices ) const noexcept;
};

mdspan<int[1][2][3]> foo = /*....*/;
auto bar = foo[0, 1, 2];
```

This syntax resembles that of a call operator, but with the proper semantics. A similar syntax can be found in popular languages such as Python, D and Ruby.

§ 4. Impact on existing code bases

We analyzed several large open source codebases and did not find a significant use of this pattern. The tool used is available on Github and automatic refactoring of large code bases is possible. In all cases, `array[x,y,z]` can be refactored in `array[(x,y,z)]` without alteration of semantics or behavior.

The codebases analyzed include the Linux kernel, Chromium, the LLVM project and the entirety of the Boost test suite. The only instances found were in Boost's "Spirit Classic" library (deprecated since 2009). A documentation of the offending syntax can be found here.

§ 5. Alternative approaches

§ 5.1. Allow comma expression only in C arrays

To maintain compatibility with C, it would be possible to deprecate the use of the comma operator only in overloaded operators. Example:

```
int c_array[1];
c_array[1,0];   // not deprecated

std::array<int, 1> cpp_array;
cpp_array[1,0];   // deprecated
```

However, this would probably lead to more confusion for a feature that is virtually unused.

§ 5.2. Alternative syntax for multidimensional subscript expressions

It has been proposed that `array[x][y]` should be equivalent to `array.operator[](x, y);`

```
mdspan<int, array_property::dimension<2,3>> foo;
auto x = foo[0][0]; //equivalent to foo(0, 0);
```

However, it's easy to imagine a scenario such that `(array[x])[y]` would be a valid expression. For example:

```
struct row {
    Foo & operator[size_t];
};
struct grid {
    row & operator[size_t] const;
    const Foo & operator[size_t, size_t] const;
};
grid g;
```

In this somewhat artificial example, `g[0][0]` and `(g[0])[0]` would be 2 valid, different expressions, which is confusing. Moreover, making this syntax compatible with pack expansion and fold expressions would certainly prove challenging.

On the other hand, this syntax mirror the usage of multidimensional C arrays and is also popular in other languages, such as Perl.

§ 5.3. Other comma expressions

Early feedback on this paper suggested we might want to deprecate comma expressions in more contexts. While the authors of this paper are not suggesting that, for completeness, we analyzed a few open source projects.

Below are tables of contexts in which comma expressions are found in various open source projects. The data was generated with the clang-tidy. It is meant to give a rough idea of the ways comma expressions are used and does not pretend to be completely accurate.

We provide two tables. The first counts each expression encountered throughout a project exactly one and as such gives an idea of the amount of code that would have to be modified to conform to some eventual deprecation.

§ 5.3.1. Without duplication caused by headers included multiple times throughout a project

Parent Expression	Boost	Chromium	Firefox	Kernel	Libreoffice	LLVM	Qt		Estimated LOC affecte
ParenExpr	958	7717	4360	72	9375	70	1494	24046	0.030%-0.08(
ForStmt	764	2916	1461	565	3275	919	2616	12516	0.016%-0.042
CompoundStmt	190	239	62	12	182	4	63	752	0.001%-0.003
ReturnStmt	160	329	53	-	53	6	92	693	0.001%-0.002
WhileStmt	69	19	4	-	66	-	14	172	
IfStmt	31	51	11	-	30	1	-	124	
ConditionalOperator	12	17	18	-	36	-	17	100	
CaseStmt	-	7	-	-	20	-	-	27	
DoStmt	1	10	1	-	1	-	-	13	
CXXStaticCastExpr	5	-	-	-	-	-	-	5	
ArraySubscriptExpr	1[1]	-	-	-	-	-	-	1[1]	
DefaultStmt	-	-	-	-	1	-	-	1	

Estimated 30 to 80 millions LOC compiled. This is a back of the envelop calculation meant to be an order of magnetude rather that a meaningful, accurate value.

§ 5.3.2. Non-Deduplicated results.

This second table represents the number of warnings that would be emitted following some eventual deprecation.

Parent Expression	Boost	Chromium	Firefox	Kernel	Libreoffice	LLVM	Qt	
ParenExpr	167038	69625	100295	924	55551	1451	6045	400929
ForStmt	106372	139460	99515	4114	12876	13343	11464	387144
ReturnStmt	55417	20353	1253	-	455	46	17636	95160
WhileStmt	11179	33	10	-	66	-	8822	20110
ConditionalOperator	11894	17	18	-	182	-	17	12128
CompoundStmt	8995	335	254	12	341	4	314	10255
IfStmt	504	54	26	-	359	1	-	944
CXXStaticCastExpr	653	-	-	-	-	-	-	653
DoStmt	18	14	34	-	1	-	-	67

Parent Expression	Boost	Chromium	Firefox	Kernel	Libreoffice	LLVM	Qt	
CaseStmt	-	7	-	-	35	-	-	42
ArraySubscriptExpr	4[1]	-	-	-	-	-	-	4[1]
DefaultStmt	-	-	-	-	4	-	-	4

1. The only instances found where in boost's "Spirit Classic" library (deprecated since 2009).A documentation of the offending syntax can be found here.↩

§ 6. Proposed Wording

In annex D, add a new subclause [depr.comma.subscript]

§ 6.1. D.﷿ Comma operator in subscript expressions

A comma expression ([expr.comma]) appearing as the *expr-or-braced-init-list of an array subscript expression ([expr.sub]) is deprecated. [Note: A comma appearing in parenthesis is not deprecated]*
[Example:

```
a[b,c];   //deprecated
a[(b,c)]; // Not deprecated; equivalent to a[c] except that b is evaluated as a discarded-v
```

]

After 7.6.1.1 [expr.comma] ¶1 ("A postfix expression followed by an expression in square brackets is a postfix expression..."), add a paragraph

[Note: In subscript expressions, the comma operator is deprecated; see [depr.comma.subscript].]

Add a paragraph after 7.6.19 [expr.sub] ¶2 ("In contexts where comma is given a special meaning, ...")

[Note: In subscript expressions, the comma operator is deprecated; see [depr.comma-subscript].]

§ 7. Acknowledgments

Thanks to Martin Hořeňovský who computed some of the statistics on comma operator usages presented in this paper.

Thanks to Titus Winters and JF Bastien who provided useful feedbacks and criticism.

§ References

§ Informative References

[P1277]
Isabella Muerte. Subscripts On Parade. URL: https://wg21.link/P1277